



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Lab Experiment- 14

Alpha-Beta Pruning Algorithm for Gaming

Aim

To implement the **Alpha-Beta Pruning algorithm** for a two-player game using Python.

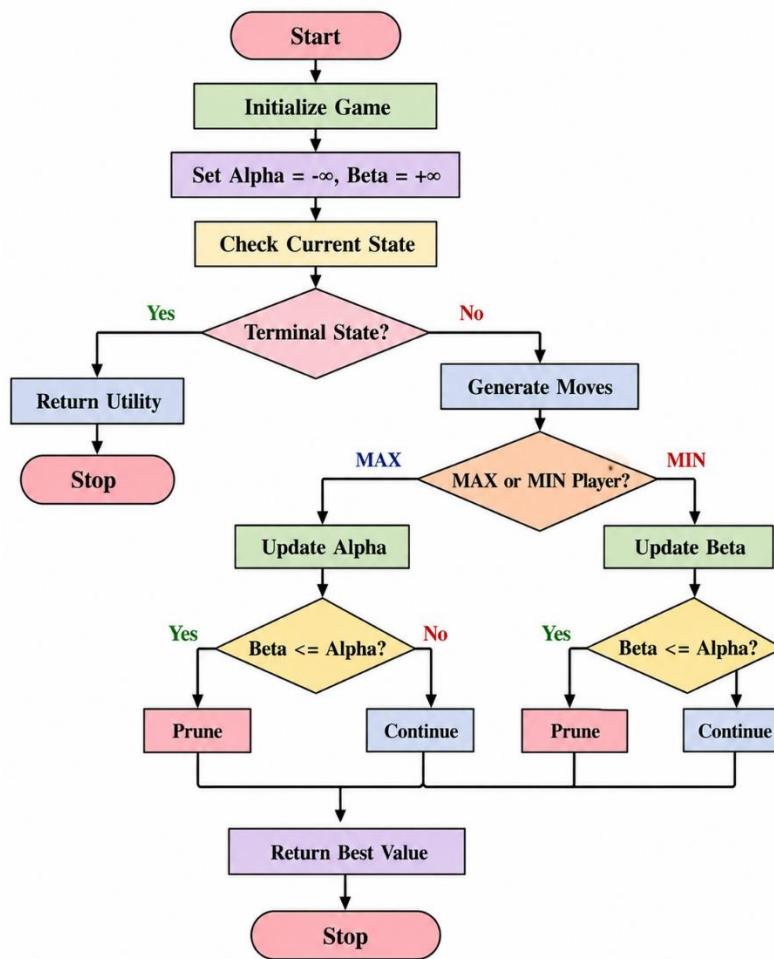
Objective

To improve the efficiency of the Minimax algorithm by eliminating branches that cannot affect the final decision.

Algorithm

1. Start the game and initialize the game tree.
2. Set $\alpha = -\infty$ and $\beta = +\infty$.
3. Check whether the current state is a terminal state.
4. Generate possible moves.
5. For the **MAX player**, select the maximum value and update α .
6. For the **MIN player**, select the minimum value and update β .
7. If $\beta \leq \alpha$, prune the remaining branches.
8. Continue until the best move is found.
9. Display the optimal value or move.
10. Stop.

FlowChart



Code

```
def alpha_beta(depth, node_index, maximizing_player,
               values, alpha, beta):

    # Terminal node
    if depth == 3:
        return values[node_index]

    if maximizing_player:
        best = float('-inf')

        for i in range(2):
            value = alpha_beta(
```

```
        depth + 1,  
        node_index * 2 + i,  
        False,  
        values,  
        alpha,  
        beta  
    )
```

```
    best = max(best, value)  
    alpha = max(alpha, best)
```

```
    # Alpha-Beta pruning  
    if beta <= alpha:  
        break
```

```
    return best
```

```
else:
```

```
    best = float('inf')
```

```
    for i in range(2):  
        value = alpha_beta(  
            depth + 1,  
            node_index * 2 + i,  
            True,  
            values,  
            alpha,  
            beta  
        )
```

```
    best = min(best, value)  
    beta = min(beta, best)
```

```

        # Alpha-Beta pruning

        if beta <= alpha:

            break

    return best

values = [3, 5, 2, 9, 12, 5, 23, 23]

```

```

alpha = float('-inf')
beta = float('inf')

```

```

result = alpha_beta(
    0, 0, True,
    values, alpha, beta
)

```

```

print("Optimal Value:", result)

```

Output

```

1  # Alpha-Beta Pruning
2
3  def alpha_beta(depth, node_index, maximizing_player,
4      values, alpha, beta):
5
6      # Terminal node
7      if depth == 3:
8          return values[node_index]
9
10     if maximizing_player:
11         best = float('-inf')
12
13         for i in range(2):
14             value = alpha_beta(
15                 depth + 1,
16                 node_index * 2 + i,
17                 False,
18                 values,
19                 alpha,
20                 beta
21             )
22
23             best = max(best, value)
24             alpha = max(alpha, best)
25
26     # Alpha-Beta pruning

```

Optimal Value: 12

```

PS C:\Users\Asus\OneDrive\Desktop\Complete-Python-3-Bootcamp-master> ^C
PS C:\Users\Asus\OneDrive\Desktop\Complete-Python-3-Bootcamp-master>
PS C:\Users\Asus\OneDrive\Desktop\Complete-Python-3-Bootcamp-master> c;; cd 'c:\Users\Asus\OneDrive\Desktop\Complete-Python-3-Bootcamp-master'; & 'C:\Users\Asus\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\Asus\vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '50646' '--' 'C:\Users\Asus\OneDrive\Desktop\Python Codes\Experiment 14.py'
Optimal Value: 12

```

Result

The Alpha-Beta Pruning algorithm was successfully implemented to find the optimal value while eliminating unnecessary branches.

Conclusion

Alpha-Beta Pruning improves the Minimax algorithm by **pruning branches that do not need to be evaluated**. It produces the same optimal result as Minimax while reducing the number of nodes examined.